FIAP · Navegando pelo mundo DevOps

Testes Automatizados de API

BDD (Gherkin) + Testes de Contrato (JSON Schema)

Projeto: Cidades ESG Inteligentes — API de Gestão de Resíduos e Reciclagem

Item	Informação
Aluno	Gustavo Ferreira Schmidt
RM	562688
Entrega	03/11/2026
Tecnologias	ASP.NET Core .NET 8 · PostgreSQL · Docker · GitHub Actions · Reqnroll + xUnit · NJsonSchema
Escopo	27 cenários Gherkin (17 positivos, 10 negativos) · 13 contratos JSON Schema · 12 rotas cobertas

Sumário

1. Introdução e objetivo
2. Projeto sob teste e arquitetura da solução de testes
3. Atendimento aos requisitos da atividade
4. Cenários BDD em Gherkin (27 cenários, detalhados)
5. Testes de contrato com JSON Schema
6. Implementação (Reqnroll, step definitions e cliente HTTP)
7. Configuração e execução (local, Docker e CI/CD)
8. Evidências de execução e bug encontrado pelos testes
9. Prints de configuração e execução
10. Conclusão

1. Introdução e objetivo

Este documento apresenta a automação de testes aplicada ao projeto Cidades ESG Inteligentes, desenvolvido nas atividades anteriores da disciplina (API REST de gestão de resíduos e reciclagem, com pipeline CI/CD). Conforme o enunciado, a atividade foi realizada sobre o projeto existente — não foi criada uma nova aplicação: o projeto de testes foi adicionado à mesma solution e ao mesmo repositório.

O objetivo é validar, de forma automatizada e repetível, o comportamento de todas as rotas da API sob três óticas complementares:

- **Comportamento (BDD):** cenários escritos em Gherkin, em português, que descrevem as regras de negócio do ponto de vista do usuário, com casos positivos e negativos.
- **Respostas HTTP:** verificação do status code e do corpo JSON (campos e valores) de cada resposta.
- **Contrato:** validação do formato do JSON retornado contra JSON Schema (draft-07), detectando alterações que quebrariam consumidores da API.

A execução é simples e documentada: um único comando (Docker) ou o tradicional dotnet test, além de um job automático no GitHub Actions que bloqueia o deploy caso algum cenário falhe.

2. Projeto sob teste e arquitetura da solução de testes

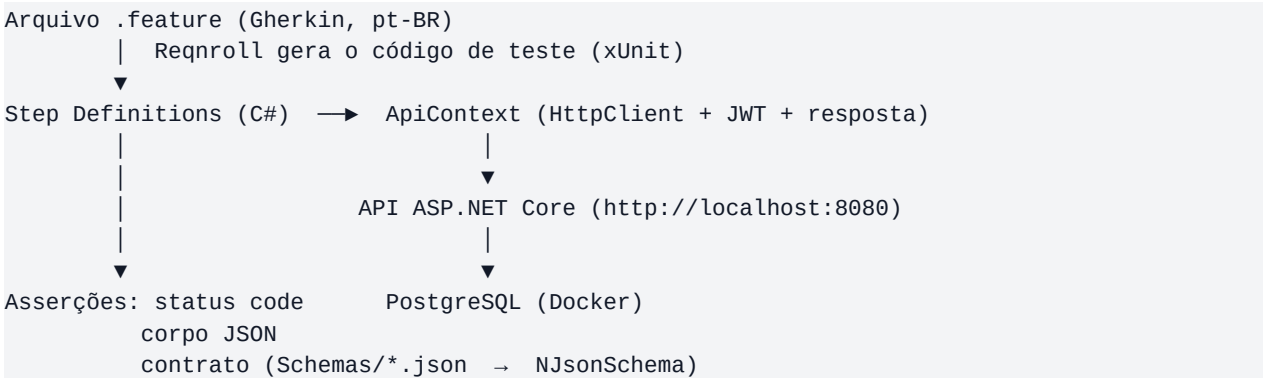
2.1 Projeto sob teste

Aspecto	Descrição
Aplicação	API REST ESG de Gestão de Resíduos (ASP.NET Core, .NET 8), com autenticação JWT
Banco de dados	PostgreSQL 16 (Docker Compose); seed com 6 pontos de coleta e 7 geradores
Regras de negócio	Tipo de resíduo compatível com o ponto; capacidade máxima; alerta automático ao atingir 80%; ponto CHEIO a 100%; atender alerta zera o ponto e o reativa
Tratamento de erros	Middleware global: NotFound→404, regra de negócio→422, argumento inválido→400, não autenticado→401; corpo no padrão ProblemDetails
Credenciais de teste	admin / Admin@123 (Administrador) e operador / Operador@123

2.2 Arquitetura da solução de testes

Os testes são caixa-preta: chamam a API real por HTTP, como um cliente faria. O endereço da API é parametrizado pela variável de ambiente API_BASE_URL (padrão http://localhost:8080), o que permite executar a mesma suíte localmente, no Docker, no CI ou contra staging.

Fluxo de execução



2.3 Estrutura de pastas

```
cidades-esg-inteligentes/
├── src/ESG.GestaoResiduos.API/                # aplicação testada
├── tests/
│   ├── ESG.GestaoResiduos.Tests/              # testes xUnit existentes (InMemory)
│   └── ESG.GestaoResiduos.BddTests/           # >>> NOVO: BDD + contrato
│       ├── Features/        (6 arquivos .feature, 27 cenários)
│       ├── StepDefinitions/ (Autenticacao, PontosColeta, Descartes,
│       │                     Alertas, Compliance, Comum)
│       ├── Schemas/         (13 contratos JSON Schema)
│       ├── Support/         (ApiContext, configuração do xUnit)
│       ├── Dockerfile       (contêiner de execução dos testes)
│       └── README.md
├── docker-compose.yml        # API + PostgreSQL
├── docker-compose.bdd.yml    # extensão: API + PostgreSQL + testes BDD
├── scripts/run-bdd-tests.sh / run-bdd-tests.ps1
└── .github/workflows/ci-cd.yml  # job "Testes BDD e contrato"
```

3. Atendimento aos requisitos da atividade

Requisito do enunciado	Como foi atendido	Onde
Utilizar o projeto das atividades anteriores	Projeto de testes adicionado à solution e ao repositório existentes	tests/ ESG.GestaoResiduos.BddTests
BDD em Gherkin — mínimo 3 cenários, positivos e negativos	27 cenários em 6 features (17 positivos e 10 negativos), em português	Seção 4 · Features/*.feature
Testes automatizados para TODAS as APIs	12 rotas cobertas: login, pontos (lista e detalhe), descartes (GET/POST), alertas (GET/PUT), dashboard, relatórios (2), health e info	Seção 4
Validação de status code	Passo “o status da resposta deve ser N” em todos os cenários (200, 201, 400, 401, 404, 422)	ComumSteps.cs
Validação do corpo JSON	Campos e valores (title, nome, alertaGerado, statusAlerta...), paginação, listas vazias / não vazias	ComumSteps.cs
Testes de contrato (JSON Schema)	13 schemas draft-07 validados com NJsonSchema em 20 cenários (tag @contrato)	Seção 5 · Schemas/
Execução documentada e simples (local ou CI)	Um comando Docker, dotnet test local e job no GitHub Actions	Seção 7 · README
ZIP com código, testes, Dockerfiles, scripts e README	ZIP entregue com toda a estrutura acima	Entrega
PDF/PPT com cenários, prints e evidências	Este documento	—

4. Cenários BDD em Gherkin (27 cenários, detalhados)

Os cenários seguem o formato Dado / Quando / Então, em português (# language: pt). Cada feature relaciona-se a um pilar ESG: Ambiental (E), Social (S) ou Governança (G). As tags classificam os cenários: @positivo, @negativo, @contrato (valida JSON Schema) e @esg (regra de negócio ESG).

Feature	Pilar	Rotas	Cenários	Pos.	Neg.
autenticacao	G	1	3	1	2
pontos_coleta	E	2	4	3	1
descartes	E / G	2	7	3	4
Alertas	E	2	6	3	3
dashboard_relatorios	G	3	5	5	0
infraestrutura	G	2	2	2	0
TOTAL		12	**27**	**17**	**10**

4.1 Autenticação de usuários

Arquivo	autenticacao.feature
Pilar ESG	Governança (G)
Rotas	POST /api/auth/login
Objetivo	Garantir que somente usuários identificados operem o sistema, assegurando rastreabilidade e segurança (JWT).

Código Gherkin — Features/autenticacao.feature

```
# language: pt

Funcionalidade: Autenticação de usuários (Governança ESG)
  Para garantir rastreabilidade e segurança nas operações de coleta de resíduos
  Como operador do sistema de gestão de resíduos
  Quero me autenticar e receber um token JWT

  @positivo @contrato
  Cenário: Login com credenciais válidas retorna um token JWT
    Dado que informo o usuário "admin" e a senha "Admin@123"
    Quando envio a requisição de login
    Então o status da resposta deve ser 200
    E a resposta deve conter um token JWT
    E o perfil retornado deve ser "Administrador"
    E a resposta deve respeitar o contrato "login-response"

  @negativo
  Cenário: Login com senha incorreta é rejeitado
    Dado que informo o usuário "admin" e a senha "SenhaErrada"
    Quando envio a requisição de login
    Então o status da resposta deve ser 422
    E o campo "title" da resposta deve ser "Regra de negócio violada"
    E a resposta deve respeitar o contrato "problem-details"

  @negativo
  Cenário: Login sem usuário e sem senha é recusado por validação
```

```
  Dado que informo o usuário "" e a senha ""
  Quando envio a requisição de login
  Então o status da resposta deve ser 400
```

O que cada cenário verifica

#	Cenário	Tipo	Verificações
1	Login com credenciais válidas retorna um token JWT	Positivo + contrato	Status 200; campo token presente; role = Administrador; corpo respeita login-response.json.
2	Login com senha incorreta é rejeitado	Negativo + contrato	Status 422 (regra de negócio); title = “Regra de negócio violada”; corpo respeita problem-details.json.
3	Login sem usuário e sem senha é recusado por validação	Negativo	Status 400 — validação de entrada (DataAnnotations) antes de qualquer regra de negócio.

4.2 Consulta de pontos de coleta

Arquivo	pontos_coleta.feature
Pilar ESG	Ambiental (E)
Rotas	GET /api/pontos-coleta · GET /api/pontos-coleta/{id}
Objetivo	Permitir que gestores e cidadãos acompanhem a ocupação dos ecopontos da cidade.

Código Gherkin — Features/pontos_coleta.feature

```
# language: pt

Funcionalidade: Consulta de pontos de coleta (Ambiental ESG)
  Para acompanhar a ocupação dos ecopontos da cidade
  Como cidadão ou gestor público
  Quero consultar os pontos de coleta de resíduos

  @positivo @contrato
  Cenário: Listar pontos de coleta retorna uma lista paginada
    Quando consulto a lista de pontos de coleta
    Então o status da resposta deve ser 200
    E a resposta deve ser paginada
    E a lista "items" não deve estar vazia
    E a resposta deve respeitar o contrato "pontos-lista"

  @positivo @contrato
  Cenário: Buscar um ponto de coleta existente retorna seus dados
    Quando consulto o ponto de coleta de id 1
    Então o status da resposta deve ser 200
    E o campo "nome" da resposta deve ser "Ecoponto Centro"
    E a resposta deve respeitar o contrato "ponto"

  @negativo @contrato
  Cenário: Buscar um ponto de coleta inexistente retorna não encontrado
    Quando consulto o ponto de coleta de id 9999
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "problem-details"

  @positivo
  Cenário: Filtrar pontos de coleta por tipo de resíduo
    Quando consulto a lista de pontos filtrando pelo tipo "PLASTICO"
    Então o status da resposta deve ser 200
```

```
    E todos os itens da lista devem ter "tipoResiduo" igual a "PLASTICO"
```

O que cada cenário verifica

#	Cenário	Tipo	Verificações
4	Listar pontos de coleta retorna uma lista paginada	Positivo + contrato	Status 200; envelope paginado (items, pageNumber, pageSize, totalItems, totalPages); items não vazio; contrato pontos-lista.json.
5	Buscar um ponto de coleta existente retorna seus dados	Positivo + contrato	Status 200; nome = “Ecoponto Centro” (id 1 do seed); contrato ponto.json.
6	Buscar um ponto de coleta inexistente retorna não encontrado	Negativo + contrato	Status 404 para o id 9999; corpo ProblemDetails (problem-details.json).
7	Filtrar pontos de coleta por tipo de resíduo	Positivo	Status 200; todos os itens retornados têm tipoResiduo = PLASTICO.

4.3 Registro de descartes de resíduos

Arquivo	descartes.feature
Pilar ESG	Ambiental (E) e Governança (G)
Rotas	GET /api/descartes · POST /api/descartes
Objetivo	Registrar cada descarte com rastreabilidade e aplicar automaticamente as regras de compliance (tipo compatível, capacidade, alerta aos 80%).

Código Gherkin — Features/descartes.feature

```
# language: pt

Funcionalidade: Registro de descartes de resíduos (Ambiental e Governança ESG)
  Para garantir a rastreabilidade do que é descartado em cada ecoponto
  Como operador autenticado do sistema
  Quero registrar descartes e ter as regras de compliance aplicadas automaticamente

  @positivo @contrato
  Cenário: Listar descartes retorna uma lista paginada
    Quando consulto a lista de descartes
    Então o status da resposta deve ser 200
    E a resposta deve ser paginada
    E a resposta deve respeitar o contrato "descartes-lista"

  @positivo @contrato
  Cenário: Operador autenticado registra um descarte válido
    Dado que estou autenticado como "operador" com a senha "Operador@123"
    Quando registro um descarte de 5 kg de "PLASTICO" no ponto 1 pelo gerador 1
    Então o status da resposta deve ser 201
    E o campo "mensagem" da resposta deve conter "sucesso"
    E a resposta deve respeitar o contrato "descarte-resultado"

  @negativo
  Cenário: Registrar descarte sem autenticação é bloqueado
    Dado que não estou autenticado
    Quando registro um descarte de 5 kg de "PLASTICO" no ponto 1 pelo gerador 1
    Então o status da resposta deve ser 401
```

```
  @negativo
  Cenário: Descarte de tipo de resíduo incompatível com o ponto é recusado
    Dado que estou autenticado como "operador" com a senha "Operador@123"
    Quando registro um descarte de 5 kg de "VIDRO" no ponto 1 pelo gerador 1
    Então o status da resposta deve ser 422
    E o campo "title" da resposta deve ser "Regra de negócio violada"
    E a resposta deve respeitar o contrato "problem-details"

  @negativo
  Cenário: Descarte em ponto de coleta inexistente retorna não encontrado
    Dado que estou autenticado como "operador" com a senha "Operador@123"
    Quando registro um descarte de 5 kg de "PLASTICO" no ponto 9999 pelo gerador 1
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "problem-details"

  @negativo
  Cenário: Descarte com peso zero é recusado por validação
    Dado que estou autenticado como "operador" com a senha "Operador@123"
    Quando registro um descarte de 0 kg de "PLASTICO" no ponto 1 pelo gerador 1
    Então o status da resposta deve ser 400

  @positivo @esg
  Cenário: Descarte que ultrapassa 80% da capacidade gera alerta automático
    Dado que estou autenticado como "operador" com a senha "Operador@123"
    E que o ponto de coleta 6 não possui alerta pendente
    Quando registro um descarte que leva o ponto 6 a 85% da capacidade, de "ELETRONICO" pelo
gerador 1
    Então o status da resposta deve ser 201
    E o campo "alertaGerado" da resposta deve ser "true"
    E existe um alerta pendente para o ponto 6
```

O que cada cenário verifica

#	Cenário	Tipo	Verificações
8	Listar descartes retorna uma lista paginada	Positivo + contrato	Status 200; envelope paginado; contrato descartes-lista.json.
9	Operador autenticado registra um descarte válido	Positivo + contrato	Login como operador; 5 kg de PLASTICO no ponto 1; status 201; mensagem contém “sucesso”; contrato descarte-resultado.json.
10	Registrar descarte sem autenticação é bloqueado	Negativo	Sem token JWT; status 401 (endpoint protegido por [Authorize]).
11	Descarte de tipo de resíduo incompatível com o ponto é recusado	Negativo + contrato	VIDRO no ponto 1 (PLASTICO); status 422; title “Regra de negócio violada”; problem-details.json.
12	Descarte em ponto de coleta inexistente retorna não encontrado	Negativo + contrato	Ponto 9999; status 404; problem-details.json.
13	Descarte com peso zero é recusado por validação	Negativo	Peso 0 kg; status 400 (validação do DTO).
14	Descarte que ultrapassa 80% da capacidade gera alerta automático	Positivo (regra ESG)	Garante ponto 6 sem alerta pendente, leva-o a 85% da capacidade; status 201; alertaGerado = true; existe alerta PENDENTE para o ponto 6.

4.4 Alertas de capacidade e atendimento de coletas

Arquivo	Alertas.feature
Pilar ESG	Ambiental (E)
Rotas	GET /api/alertas · PUT /api/alertas/{id}/atender
Objetivo	Permitir que a equipe aja antes de um ecoponto transbordar e registre a coleta realizada, liberando a capacidade do ponto.

Código Gherkin — Features/Alertas.feature

```
# language: pt

Funcionalidade: Alertas de capacidade e atendimento de coletas (Ambiental ESG)
  Para que a equipe de coleta aja antes de um ecoponto transbordar
  Como operador autenticado do sistema
  Quero consultar os alertas e registrar a coleta realizada

  @positivo @contrato
  Cenário: Listar alertas retorna uma lista paginada
    Quando consulto a lista de alertas
    Então o status da resposta deve ser 200
    E a resposta deve ser paginada
    E a resposta deve respeitar o contrato "alertas-lista"

  @positivo
  Cenário: Filtrar alertas pendentes retorna somente alertas pendentes
    Dado que estou autenticado como "operador" com a senha "Operador@123"
    E que o ponto de coleta 5 possui um alerta pendente
    Quando consulto os alertas com status "PENDENTE"
    Então o status da resposta deve ser 200
    E todos os itens da lista devem ter "statusAlerta" igual a "PENDENTE"

  @positivo @esg @contrato
  Cenário: Atender um alerta registra a coleta e libera a capacidade do ponto
    Dado que estou autenticado como "operador" com a senha "Operador@123"
    E que o ponto de coleta 5 possui um alerta pendente
    Quando atendo o alerta pendente
    Então o status da resposta deve ser 200
    E o campo "statusAlerta" da resposta deve ser "ATENDIDO"
    E o campo "statusPonto" da resposta deve ser "ATIVO"
    E a quantidade atual do ponto deve ser zero
    E a resposta deve respeitar o contrato "atender-alerta-response"

  @negativo
  Cenário: Atender alerta sem autenticação é bloqueado
    Dado que não estou autenticado
    Quando atendo o alerta de id 1
    Então o status da resposta deve ser 401

  @negativo
  Cenário: Atender alerta inexistente retorna não encontrado
    Dado que estou autenticado como "operador" com a senha "Operador@123"
    Quando atendo o alerta de id 999999
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "problem-details"

  @negativo
  Cenário: Atender um alerta já atendido é recusado
    Dado que estou autenticado como "operador" com a senha "Operador@123"
```

```
    E que o ponto de coleta 5 possui um alerta pendente
    Quando atendo o alerta pendente
    E atendo novamente o mesmo alerta
    Então o status da resposta deve ser 422
    E o campo "title" da resposta deve ser "Regra de negócio violada"
    E a resposta deve respeitar o contrato "problem-details"
```

O que cada cenário verifica

#	Cenário	Tipo	Verificações
15	Listar alertas retorna uma lista paginada	Positivo + contrato	Status 200; envelope paginado; contrato alertas-lista.json.
16	Filtrar alertas pendentes retorna somente alertas pendentes	Positivo	Prepara um alerta pendente no ponto 5; filtra status = PENDENTE; todos os itens têm statusAlerta = PENDENTE.
17	Atender um alerta registra a coleta e libera a capacidade do ponto	Positivo (regra ESG) + contrato	Status 200; statusAlerta = ATENDIDO; statusPonto = ATIVO; quantidade atual do ponto = 0; contrato atender-alerta-response.json.
18	Atender alerta sem autenticação é bloqueado	Negativo	Status 401.
19	Atender alerta inexistente retorna não encontrado	Negativo + contrato	Alerta 999999; status 404; problem-details.json.
20	Atender um alerta já atendido é recusado	Negativo + contrato	Atende o mesmo alerta duas vezes; segunda chamada: status 422; “Regra de negócio violada”; problem-details.json.

4.5 Indicadores ESG e relatórios de compliance

Arquivo	dashboard_relatorios.feature
Pilar ESG	Governança (G)
Rotas	GET /api/dashboard · GET /api/relatorios-compliance · GET /api/relatorios-compliance/consolidado
Objetivo	Permitir ao gestor público acompanhar a conformidade ambiental da operação por meio de indicadores e relatórios mensais.

Código Gherkin — Features/dashboard_relatorios.feature

```
# language: pt

Funcionalidade: Indicadores ESG e relatórios de compliance (Governança ESG)
  Para acompanhar a conformidade ambiental da operação
  Como gestor público
  Quero consultar o dashboard e os relatórios de compliance

  @positivo @contrato
  Cenário: Dashboard retorna os indicadores consolidados
    Quando consulto o dashboard
    Então o status da resposta deve ser 200
    E o campo "totalPontos" da resposta deve ser "6"
    E a resposta deve respeitar o contrato "dashboard-response"

  @positivo @contrato
  Cenário: Listar relatórios de compliance retorna uma lista paginada
    Quando consulto a lista de relatórios de compliance
    Então o status da resposta deve ser 200
```

```
    E a resposta deve ser paginada
    E a resposta deve respeitar o contrato "relatorios-lista"

  @positivo
  Cenário: Relatórios de um ano sem dados retornam lista vazia
    Quando consulto os relatórios de compliance do ano 1900
    Então o status da resposta deve ser 200
    E a lista "items" deve estar vazia
    E o campo "totalItems" da resposta deve ser "0"

  @positivo @esg @contrato
  Cenário: Descarte registrado aparece no consolidado de compliance
    Dado que existe ao menos um descarte registrado
    Quando consulto o consolidado de compliance
    Então o status da resposta deve ser 200
    E a resposta deve ser uma lista não vazia
    E a resposta deve respeitar o contrato "relatorios-consolidado"

  @positivo
  Cenário: Consolidado de um ano sem dados retorna lista vazia
    Quando consulto o consolidado de compliance do ano 1900
    Então o status da resposta deve ser 200
    E a resposta deve ser uma lista vazia
```

O que cada cenário verifica

#	Cenário	Tipo	Verificações
21	Dashboard retorna os indicadores consolidados	Positivo + contrato	Status 200; totalPontos = 6 (seed); contrato dashboard-response.json.
22	Listar relatórios de compliance retorna uma lista paginada	Positivo + contrato	Status 200; envelope paginado; contrato relatorios-lista.json.
23	Relatórios de um ano sem dados retornam lista vazia	Positivo	Filtro ano = 1900; status 200; items vazio; totalItems = 0.
24	Descarte registrado aparece no consolidado de compliance	Positivo (regra ESG) + contrato	Garante ao menos um descarte; status 200; resposta é uma lista não vazia; contrato relatorios-consolidado.json. (Este cenário revelou o bug de HTTP 500 — ver seção 8.)
25	Consolidado de um ano sem dados retorna lista vazia	Positivo	Ano 1900; status 200; lista vazia. (Também revelou o bug — seção 8.)

4.6 Saúde e rastreabilidade do deploy

Arquivo	infraestrutura.feature
Pilar ESG	Governança (G)
Rotas	GET /health · GET /api/info
Objetivo	Saber se a API e o banco estão saudáveis e qual versão do software está em execução.

Código Gherkin — Features/infraestrutura.feature

```
# language: pt

Funcionalidade: Saúde e rastreabilidade do deploy (Governança ESG)
  Para garantir que a aplicação está no ar e saber exatamente qual versão está rodando
  Como equipe de operação
```

```
Quero consultar a saúde da API e as informações do ambiente

@positivo @contrato
Cenário: Health check informa que a API e o banco estão saudáveis
  Quando consulto o endpoint "/health"
  Então o status da resposta deve ser 200
  E o campo "status" da resposta deve ser "Healthy"
  E a resposta deve respeitar o contrato "health-response"

@positivo @contrato
Cenário: Info informa o ambiente e a versão em execução
  Quando consulto o endpoint "/api/info"
  Então o status da resposta deve ser 200
  E a resposta deve respeitar o contrato "info-response"
```

O que cada cenário verifica

#	Cenário	Tipo	Verificações
26	Health check informa que a API e o banco estão saudáveis	Positivo + contrato	Status 200; status = Healthy; contrato health-response.json.
27	Info informa o ambiente e a versão em execução	Positivo + contrato	Status 200; contrato info-response.json.

5. Testes de contrato com JSON Schema

Um teste de contrato garante que o formato da resposta (nomes dos campos, tipos, obrigatoriedade e valores permitidos) continua o esperado pelos consumidores da API. Se um campo obrigatório for removido, um tipo for alterado ou um valor de enum mudar, o cenário falha indicando o caminho do campo e a regra violada.

Para isso, cada resposta relevante é validada contra um arquivo JSON Schema (draft-07), usando a biblioteca NJsonSchema, por meio do passo reutilizável:

Passo Gherkin

```
E a resposta deve respeitar o contrato "login-response"
```

Contrato (Schemas/*.json)	Rota	Estrutura validada
login-response.json	POST /api/auth/login	token, tipo (=Bearer), expiraEm (date-time), username, role
problem-details.json	Respostas de erro (400/404/422/500)	title, status (400–599), detail, instance
pontos-lista.json	GET /api/pontos-coleta	Envelope paginado + itens de ponto
ponto.json	GET /api/pontos-coleta/{id}	id, nome, tipoResiduo, capacidade, status, ocupação
descartes-lista.json	GET /api/descartes	Envelope paginado + itens de descarte
descarte-resultado.json	POST /api/descartes	mensagem, alertaGerado, dados do descarte/ponto
alertas-lista.json	GET /api/alertas	Envelope paginado + itens de alerta (statusAlerta enum)
atender-alerta-response.json	PUT /api/alertas/{id}/atender	statusAlerta, statusPonto, quantidade atual
dashboard-response.json	GET /api/dashboard	totalPontos e indicadores consolidados
relatorios-lista.json	GET /api/relatorios-compliance	Envelope paginado + relatórios mensais
relatorios-consolidado.json	GET /api/relatorios-compliance/consolidado	Lista por período (ano/mês, kg, % conformidade)
health-response.json	GET /health	status = Healthy e checks
info-response.json	GET /api/info	ambiente e versão

5.1 Exemplos de contratos

Schemas/login-response.json

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "LoginResponse",
  "type": "object",
  "required": ["token", "tipo", "expiraEm", "username", "role"],
  "properties": {
    "token":    { "type": "string", "minLength": 20 },
    "tipo":     { "type": "string", "enum": ["Bearer"] },
    "expiraEm": { "type": "string", "format": "date-time" },
    "username": { "type": "string", "minLength": 1 },
    "role":     { "type": "string", "minLength": 1 }
  },
```

```
  "additionalProperties": false
}
```

Schemas/problem-details.json (formato padrão de todos os erros)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ProblemDetails",
  "type": "object",
  "required": ["title", "status", "detail", "instance"],
  "properties": {
    "type":     { "type": ["string", "null"] },
    "title":    { "type": "string", "minLength": 1 },
    "status":   { "type": "integer", "minimum": 400, "maximum": 599 },
    "detail":   { "type": "string", "minLength": 1 },
    "instance": { "type": "string" }
  }
}
```

5.2 Como o contrato é verificado no código

StepDefinitions/ComumSteps.cs

```
[Then("a resposta deve respeitar o contrato {string}")]
public async Task ARespostaDeveRespeitarOContrato(string nomeDoContrato)
{
    var caminho = Path.Combine(AppContext.BaseDirectory, "Schemas",
$"{nomeDoContrato}.json");
    Assert.True(File.Exists(caminho), $"Arquivo de contrato não encontrado: {caminho}");

    var schema = await JsonSchema.FromJsonAsync(await File.ReadAllTextAsync(caminho));
    var violacoes = schema.Validate(_ctx.ResponseBody);

    Assert.True(violacoes.Count == 0,
        "Violações do contrato: " + string.Join("; ", violacoes.Select(v => $"{v.Path}
({v.Kind})")));
}
```

> **Experimento sugerido (prova de que o contrato realmente protege a API)**
>
> 1) Em Schemas/login-response.json troque o enum de tipo de ["Bearer"] para ["Basic"].
>
> 2) Rode somente o cenário de login: dotnet test --filter "FullyQualifiedName~Login".
>
> 3) O teste falha com “Violações do contrato: #/tipo (NotInEnumeration)”. Tire o print (seção 9, P6) e desfaça a alteração.

6. Implementação (Reqnroll, step definitions e cliente HTTP)

6.1 Pacotes utilizados

Pacote	Versão	Finalidade
Reqnroll.xUnit	3.3.4	BDD: lê os arquivos .feature (Gherkin) e gera os testes xUnit; sucessor do SpecFlow
xunit / xunit.runner.visualstudio	2.9.0 / 2.5.3	Framework e executor de testes
Microsoft.NET.Test.Sdk	17.8.0	Integração com dotnet test
NJsonSchema	11.6.1	Validação de JSON Schema (testes de contrato)
coverlet.collector	6.0.0	Cobertura de código

ESG.GestaoResiduos.BddTests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>

    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="coverlet.collector" Version="6.0.0" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.8.0" />
    <PackageReference Include="NJsonSchema" Version="11.6.1" />
    <PackageReference Include="Reqnroll.xUnit" Version="3.3.4" />
    <PackageReference Include="xunit" Version="2.9.0" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.5.3" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="Xunit" />
  </ItemGroup>

  <ItemGroup>
    <None Update="Schemas\**\*.json">
      <CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory>
    </None>
  </ItemGroup>

</Project>
```

6.2 Cliente HTTP do cenário (ApiContext)

O Reqnroll cria uma instância de ApiContext por cenário (injeção de contexto), garantindo isolamento: cada cenário tem seu próprio HttpClient, token JWT e última resposta. A URL base vem de API_BASE_URL.

Support/ApiContext.cs (resumo)

```
public class ApiContext : IDisposable
{
    public static readonly string BaseUrl =
```

```
        Environment.GetEnvironmentVariable("API_BASE_URL") ?? "http://localhost:8080";

    public string? Token { get; set; }          // JWT obtido no login
    public HttpResponseMessage? Response { get; private set; }
    public string ResponseBody { get; private set; } = string.Empty;
    public JsonDocument? Json { get; private set; }
    public int StatusCode => (int)(Response?.StatusCode ?? 0);

    public Task GetAsync(string caminho)  => EnviarAsync(HttpMethod.Get,  caminho, null);
    public Task PostAsync(string caminho, object corpo) => EnviarAsync(HttpMethod.Post,
caminho, corpo);
    public Task PutAsync(string caminho)  => EnviarAsync(HttpMethod.Put,  caminho, null);
    // ... adiciona "Authorization: Bearer <token>" quando há token
}
```

6.3 Decisões de projeto

- **Cenários independentes:** cada cenário prepara seus próprios dados (ex.: cria/zera um alerta no ponto 5 ou 6), sem depender da ordem de execução.
- **Sem paralelismo:** [assembly: CollectionBehavior(DisableTestParallelization = true)], pois os cenários compartilham o mesmo banco.
- **Passos reutilizáveis:** ComumSteps concentra status code, campos, paginação, listas e contrato; steps específicos ficam por funcionalidade.
- **Mensagens de falha úteis:** o status inesperado exibe também o corpo da resposta, o que acelerou o diagnóstico do bug da seção 8.

7. Configuração e execução (local, Docker e CI/CD)

7.1 Execução com um comando (Docker — recomendada)

O script sobe PostgreSQL + API + testes, espera a API ficar saudável, executa todos os cenários, grava o relatório TRX em TestResults-BDD/, derruba o ambiente e devolve o código de saída dos testes.

Comandos

```
# Linux / macOS / Git Bash
bash scripts/run-bdd-tests.sh

# Windows PowerShell
.\scripts\run-bdd-tests.ps1

# Equivalente manual
cp .env.example .env
docker compose -f docker-compose.yml -f docker-compose.bdd.yml up --build --abort-on-
container-exit --exit-code-from bdd-tests
docker compose -f docker-compose.yml -f docker-compose.bdd.yml down -v
```

7.2 Execução local com dotnet test

Comandos

```
cp .env.example .env                  # PowerShell: Copy-Item .env.example .env
docker compose up -d --build          # API + PostgreSQL em http://localhost:8080
docker compose ps                     # aguardar "healthy"

dotnet test tests/ESG.GestaoResiduos.BddTests/ESG.GestaoResiduos.BddTests.csproj

# outra URL:  $env:API_BASE_URL="https://minha-api"   (PowerShell)
```

7.3 Dockerfile dos testes

tests/ESG.GestaoResiduos.BddTests/Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS tests
WORKDIR /src

COPY tests/ESG.GestaoResiduos.BddTests/ESG.GestaoResiduos.BddTests.csproj
tests/ESG.GestaoResiduos.BddTests/
RUN dotnet restore tests/ESG.GestaoResiduos.BddTests/ESG.GestaoResiduos.BddTests.csproj

COPY tests/ESG.GestaoResiduos.BddTests/ tests/ESG.GestaoResiduos.BddTests/
RUN dotnet build tests/ESG.GestaoResiduos.BddTests/ESG.GestaoResiduos.BddTests.csproj -c
Release --no-restore

ENV API_BASE_URL=http://api:8080
ENV DOTNET_NOLOGO=true

ENTRYPOINT ["dotnet", "test",
"tests/ESG.GestaoResiduos.BddTests/ESG.GestaoResiduos.BddTests.csproj", \
            "-c", "Release", "--no-build", \
            "--logger", "trx;LogFileName=resultado-bdd.trx", \
            "--logger", "console;verbosity=normal", \
            "--results-directory", "/results"]
```

7.4 docker-compose.bdd.yml

docker-compose.bdd.yml (extensão do docker-compose.yml)

```
services:
  bdd-tests:
    build:
      context: .
      dockerfile: tests/ESG.GestaoResiduos.BddTests/Dockerfile
    image: cidades-esg-bdd-tests:local
    depends_on:
      api:
        condition: service_healthy      # só começa quando a API está "healthy"
    environment:
      API_BASE_URL: http://api:8080
    volumes:
      - ./TestResults-BDD:/results
    networks:
      - frontend
```

7.5 Pipeline CI/CD (GitHub Actions)

O workflow .github/workflows/ci-cd.yml ganhou o job “2. Testes BDD (Gherkin) e contrato (JSON Schema)”. Ele roda em todo push e pull request, sobe API + PostgreSQL com Docker Compose, aguarda /health, executa os 27 cenários com API_BASE_URL=http://localhost:8080 e publica o resumo na aba Summary e os artefatos resultados-bdd (.trx) e docker-compose-bdd-logs. O deploy em staging depende desse job (needs: [docker, bdd-tests]); se qualquer cenário falhar, nada é publicado.

Fluxo dos jobs

```
build-test  ──►  bdd-tests (Gherkin + contrato)  ──┐
     │                                              ├──►  deploy-staging  ──►  deploy-
production
     └──────────►  docker (build, compose, GHCR)  ──┘
```

ci-cd.yml — trecho do job bdd-tests

```
  bdd-tests:
    name: "2. Testes BDD (Gherkin) e contrato (JSON Schema)"
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Instalar .NET SDK
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: ${{ env.DOTNET_VERSION }}

      - name: Subir ambiente (API + PostgreSQL) com Docker Compose
        env:
          ASPNETCORE_ENVIRONMENT: Staging
        run: |
          cp .env.example .env
          docker compose up -d --build
          docker compose ps

      - name: Aguardar a API ficar saudável
        run: |
          for i in $(seq 1 60); do
            if curl -fsS http://localhost:8080/health > /dev/null 2>&1; then
```

```
              echo "API respondeu em ${i} tentativa(s)."; exit 0
            fi
            sleep 3
          done
          echo "::error::A API não ficou saudável a tempo."
          docker compose logs --no-color api | tail -n 80
          exit 1

      - name: Executar testes BDD e de contrato
        env:
          API_BASE_URL: http://localhost:8080
        run: >
          dotnet test tests/ESG.GestaoResiduos.BddTests/ESG.GestaoResiduos.BddTests.csproj -
c Release
          --logger "trx;LogFileName=resultado-bdd.trx"
          --logger "console;verbosity=normal"
          --results-directory ./TestResults-BDD
```

Observação: o job build-test e o estágio test do Dockerfile principal passaram a executar apenas o projeto xUnit existente (tests/ESG.GestaoResiduos.Tests), pois os testes BDD dependem de uma API em execução e rodam no job dedicado.

8. Evidências de execução e bug encontrado pelos testes

8.1 Evolução das execuções

A suíte foi construída de forma incremental, executando dotnet test a cada etapa. Os resultados abaixo foram obtidos nas execuções reais no computador do aluno (.NET 8.0.28, Windows):

Etapa	Conteúdo adicionado	Resultado
1	Esqueleto do projeto e 7 cenários (autenticação e pontos de coleta) — passos ainda não implementados	7 cenários descobertos (pendentes)
2	Passos de autenticação e pontos de coleta	7/7 passando
3	Feature de descartes (7 cenários)	14/14 passando
4	Feature de alertas (6 cenários)	20/20 passando
5	Dashboard, relatórios e infraestrutura (7 cenários) + contratos JSON Schema	25 passaram · 2 falharam (27 total)
6	Correção do bug de HTTP 500 no consolidado (seção 8.3) e nova execução	27/27 esperado — ver print P5 (seção 9)

8.2 Execução real que detectou o defeito (25 de 27)

Transcrição do terminal (trecho) da execução da etapa 5. Os dois cenários que falharam eram justamente os que consultam o endpoint de consolidado de compliance — os testes automatizados apontaram um defeito real da API, que passava despercebido nos testes anteriores:

Saída do terminal (transcrição da execução real)

```
PS ...\tests\ESG.GestaoResiduos.BddTests> dotnet test
[xUnit.net 00:00:01.87]     Descarte registrado aparece no consolidado de compliance [FAIL]
[xUnit.net 00:00:01.87]       Esperado status 200, mas a API retornou 500. Corpo:
        {"title":"Erro interno do servidor","status":500,
         "detail":"Ocorreu um erro inesperado. Contate o suporte.",
         "instance":"/api/relatorios-compliance/consolidado"}
...
        Dado que existe ao menos um descarte registrado
        -> done: ComplianceSteps.QueExisteAoMenosUmDescarteRegistrado() (0.1s)
        Quando consulto o consolidado de compliance
        -> done: ComplianceSteps.ConsultoOConsolidado() (0.1s)
        Então o status da resposta deve ser 200
        -> error: Esperado status 200, mas a API retornou 500.
        E a resposta deve ser uma lista não vazia
        -> skipped because of previous errors
        E a resposta deve respeitar o contrato "relatorios-consolidado"
        -> skipped because of previous errors
[xUnit.net 00:00:01.96]     Consolidado de um ano sem dados retorna lista vazia [FAIL]
...
Resumo do teste: total: 27, falhou: 2, bem-sucedido: 25, ignorado: 0, duração: 3.8s
Construir falhou com 2 erro(s) em 8.5s
```

8.3 Diagnóstico e correção

O log do contêiner da API (docker compose logs api) revelou a causa:

Log da API (docker compose logs api)

```
Npgsql.PostgresException (0x80004005): 42804: argument of FILTER must be type boolean,
```

```
not type character varying
```

src/ESG.GestaoResiduos.API/Services/RelatorioService.cs

```
// ANTES (RelatorioService.ConsolidarAsync) — gera COUNT(*) FILTER (WHERE "CONFORME") no
PostgreSQL
.GroupBy(r => new { r.AnoReferencia, r.MesReferencia })
.Select(g => new {
    ...,
    QtdConformes    = g.Count(x => x.Conforme),
    QtdNaoConformes = g.Count(x => !x.Conforme) })

// DEPOIS — Conforme entra na chave do GROUP BY e a união por período é feita em memória
var grupos = await query
    .GroupBy(r => new { r.AnoReferencia, r.MesReferencia, r.Conforme })
    .Select(g => new { g.Key.AnoReferencia, g.Key.MesReferencia, g.Key.Conforme,
        TotalKg = g.Sum(x => x.TotalKgColetado), QtdDescartes = g.Sum(x => x.QtdDescartes),
        QtdRelatorios = g.Count() })
    .ToListAsync(ct);

return grupos.GroupBy(g => new { g.AnoReferencia, g.MesReferencia })
    .Select(periodo => {
        var qtdRelatorios = periodo.Sum(x => x.QtdRelatorios);
        var qtdConformes  = periodo.Where(x => x.Conforme).Sum(x => x.QtdRelatorios);
        return new RelatorioConsolidadoResponse { ...,
            QtdNaoConformes = qtdRelatorios - qtdConformes,
            PercentualConformidade = qtdRelatorios > 0
                ? Math.Round((decimal)qtdConformes / qtdRelatorios * 100m, 2) : 0m };
    })
    .OrderByDescending(r => r.AnoReferencia).ThenByDescending(r => r.MesReferencia)
    .ToList();
```

Esse é o valor prático da automação: os cenários BDD, executados contra a API real com PostgreSQL, encontraram um defeito que os 12 testes xUnit existentes (que usam banco InMemory, onde a conversão S/N não é traduzida para SQL) não conseguiam detectar. Após a correção e a reconstrução da imagem (docker compose up -d --build), a suíte completa deve apresentar 27 de 27 cenários aprovados (print P5).

9. Prints de configuração e execução

📷 **P1 — COLE AQUI O PRINT**

Estrutura do projeto de testes

Árvore de pastas de tests/ESG.GestaoResiduos.BddTests com Features, Schemas, StepDefinitions, Support, Dockerfile e README (IntelliJ ou Explorer).

📷 **P2 — COLE AQUI O PRINT**

Pacotes do projeto de testes

Arquivo ESG.GestaoResiduos.BddTests.csproj aberto, mostrando Reqnroll.xUnit, xunit e NJsonSchema.

📷 **P3 — COLE AQUI O PRINT**

Cenário Gherkin

Um arquivo .feature aberto no editor (ex.: descartes.feature), mostrando Cenário, Dado/Quando/Então e as tags.

📷 **P4 — COLE AQUI O PRINT**

Ambiente no ar (Docker Compose)

Terminal com os serviços db e api em estado healthy.

```
docker compose ps
```

📷 **P5 — COLE AQUI O PRINT**

Execução da suíte — 27 de 27 aprovados

Terminal mostrando a linha final “total: 27, falhou: 0, bem-sucedido: 27”.

`dotnet test tests/ESG.GestaoResiduos.BddTests/ESG.GestaoResiduos.BddTests.csproj`

📷 **P6 — COLE AQUI O PRINT**

Falha provocada de contrato (experimento da seção 5)

Teste de login falhando com “Violações do contrato: #/tipo (NotInEnumeration)” depois de trocar Bearer por Basic no schema (lembre de desfazer).

`dotnet test --filter "FullyQualifiedName~Login"`

📷 **P7 — COLE AQUI O PRINT**

Execução via Docker (um comando)

Saída do script que sobe API + banco + testes e termina com sucesso (opcional, mas recomendado).

`bash scripts/run-bdd-tests.sh` (PowerShell: `.\scripts\run-bdd-tests.ps1`)

📷 **P8 — COLE AQUI O PRINT**

Pipeline no GitHub Actions

Página da execução do workflow com os jobs verdes, incluindo “2. Testes BDD (Gherkin) e contrato (JSON Schema)”, e a seção Summary/Artifacts (resultados-bdd).

`GitHub › Actions › última execução`

10. Conclusão

A atividade foi concluída sobre o projeto Cidades ESG Inteligentes, sem criar uma nova aplicação. Foram automatizados 27 cenários BDD em Gherkin (17 positivos e 10 negativos) que cobrem as 12 rotas da API, validando status code, corpo JSON e contrato por JSON Schema (13 contratos). A execução é simples — um comando com Docker, dotnet test local ou o job do GitHub Actions — e está documentada no README.

Os testes demonstraram valor imediato: identificaram um defeito real (HTTP 500 no consolidado de compliance por incompatibilidade entre o value converter S/N e o PostgreSQL), que foi diagnosticado pelos logs e corrigido. Do ponto de vista ESG, os cenários protegem regras de negócio dos três eixos: Ambiental (capacidade dos ecopontos, alertas e coletas), Governança (autenticação, rastreabilidade, compliance e saúde do deploy) e, de forma indireta, Social (transparência dos dados para gestores e cidadãos).

Próximos passos possíveis

- Adicionar relatório HTML de BDD (por exemplo, Reqnroll LivingDoc) como artefato do pipeline.
- Executar a suíte contra staging após cada deploy (API_BASE_URL apontando para o Render).
- Ampliar cenários negativos de validação (limites de peso, campos obrigatórios) e de autorização por perfil.